

Week 3

Libraries and Tools Used

The code makes use of TensorFlow as the primary deep learning framework for building and training neural network models. NumPy is used for efficient numerical operations, while Matplotlib is employed to visualize datasets, training behavior, and model performance. Scikit-learn is utilized for loading housing datasets, splitting data into training and testing sets, feature normalization, and computing evaluation metrics such as mean absolute error and R^2 score. Together, these tools enable efficient data handling, model development, and result analysis.

Custom Neural Network Layers

To expose the internal mechanics of neural networks, the program defines custom layers by subclassing the TensorFlow `Layer` class. A custom flatten layer reshapes image inputs into vectors suitable for fully connected layers. A custom dense layer with ReLU activation is implemented by explicitly defining weight and bias parameters and applying a linear transformation followed by the ReLU activation function. Another custom dense layer uses Softmax activation to convert raw outputs into class probability distributions. These implementations replicate standard neural network operations while allowing greater control and transparency.

MNIST

The MNIST dataset is loaded using TensorFlow's built-in dataset utilities. The input images are normalized so that pixel values lie between 0 and 1, which improves training stability. The digit labels are converted into one-hot encoded vectors to support multi-class classification. Sample images from the dataset are visualized and saved to provide a clear illustration of the handwritten digit inputs. A neural network model for digit classification is constructed using the custom flatten, ReLU, and Softmax layers. The model is compiled with the Adam optimizer and categorical cross-entropy loss function. Training is performed over multiple epochs with a validation split to monitor generalization performance. After training, the model is evaluated on unseen test data, and classification accuracy is reported to assess the effectiveness of the custom-layer-based architecture.

Housing Dataset

For the regression task, the code loads a housing price dataset, using the Boston Housing dataset when available or the California Housing dataset as an alternative. The dataset is divided into training and testing subsets to allow unbiased evaluation. All numerical features are standardized using a z-score normalization technique, ensuring that differences in feature scales do not adversely affect the learning process.

Linear Regression Model

A baseline linear regression model is implemented using a single-neuron neural network without an activation function. This model serves as a reference point for evaluating more complex architectures. The model is trained using mean squared error as the loss function and mean absolute error as an additional metric. Its performance on the test dataset provides a benchmark against which neural network models are compared.

Neural Network Regression Experiments

Several neural network architectures with varying depths, layer sizes, and activation functions are trained to predict housing prices. These include shallow, medium, and deep networks, as well as a network using a different activation function. All models are trained under identical conditions to ensure a fair comparison. Their performance metrics are recorded to analyze how network complexity influences regression accuracy.

Results and Performance Comparison

The results from both classification and regression experiments are summarized and compared. The MNIST classification accuracy demonstrates the effectiveness of the custom layers. For the housing dataset, neural network models are compared against the linear regression baseline, and percentage improvements are computed to quantify performance gains achieved through deeper architectures.

Visualization and Analysis

The program generates multiple plots to visualize training behavior and model performance. These include accuracy curves for MNIST classification, loss curves for regression models, bar charts comparing error metrics across architectures, and scatter plots comparing predicted versus actual housing prices. An R^2 score is calculated and displayed to further evaluate the predictive quality of the best-performing regression model.

Week 4

Objective

The goal of this project is to build a character-level language model that can learn and generate text sequences. Unlike word-based models, character-level models work directly with individual characters, allowing them to learn spelling, grammar, and writing style without relying on predefined vocabularies. This makes the model flexible and capable of handling rare or unseen words.

Dataset

The model is trained on a text corpus containing 1,115,394 characters with 65 unique characters in total. The data is split into 90% training data and 10% test data. This split ensures that the model is evaluated on unseen text and helps measure how well it generalizes beyond the training set.

The project uses TensorFlow and Keras for model implementation, a Transformer-based architecture for sequence modeling, character-level tokenization for text representation, and autoregressive generation to produce text one character at a time.

2. Model Architecture

Overall Design

The model follows a decoder-only Transformer architecture, similar to GPT-style models. It processes fixed-length character sequences and predicts the next character at each position.

Token and Position Embedding

Each character is first converted into a dense vector using token embeddings. Positional embeddings are added to give the model information about the order of characters in the sequence. These embeddings together allow the model to understand both what each character is and where it appears. The model contains four Transformer blocks. Each block includes multi-head self-attention, a feed-forward neural network, layer normalization, residual connections, and dropout. Self-attention allows the model to focus on different parts of the sequence, while residual connections and normalization help stabilize training. A causal attention mask is used to ensure that the model can only attend to past and current characters, not future ones. This is essential for autoregressive text generation and prevents the model from using information it should not have access to during training. A final dense layer maps the model's internal representations to the vocabulary size, producing logits for each possible next character.

3. Technical Implementation

The text data is loaded and minimally preprocessed by replacing newline characters with spaces. A vocabulary is built by identifying all unique characters and mapping them to numerical indices. The

data is then converted into training examples using a sliding window approach, where each input sequence of 128 characters is paired with a target sequence shifted by one character. The model is compiled using the Adam optimizer and sparse categorical cross-entropy loss. Accuracy is used as an additional metric to track how often the model predicts the correct next character. Text generation is performed autoregressively. Starting from a seed sequence, the model predicts the next character, samples from the probability distribution, appends the character to the sequence, and repeats this process to generate longer text outputs.

4. Training Process

The model was trained with a batch size of 32 and a context window of 128 characters. Due to hardware limitations, training was slow, taking approximately two seconds per batch. As a result, only about 10% of the first epoch was completed. Despite this, the model achieved around 63% accuracy and a loss of approximately 1.27, which is reasonable performance for a character-level model with 65 possible outputs.

5. Analysis and Reflection

Strengths

The implementation is modular and well-structured, with clear separation between data processing, model definition, and training. Modern best practices such as residual connections, layer normalization, and dropout are used correctly. The chosen hyperparameters balance model capacity and computational feasibility, making the project suitable for a learning-focused implementation.

Areas for Improvement

Training speed is the biggest limitation. Enabling GPU acceleration would significantly reduce training time. Adding model checkpointing would allow training to resume after interruptions. Including validation during training would help monitor overfitting, and learning rate scheduling could further improve convergence. More advanced text generation methods, such as temperature or top-k sampling, could improve output quality.

Conclusion

This project required a an understanding of how attention mechanisms work, particularly causal masking for autoregressive models. Balancing model complexity with limited computational resources was a major challenge. The project highlighted the importance of efficient data pipelines and proper infrastructure when working with deep learning models.

Key learnings include a strong appreciation for the power of transformer architectures, insights into character-level language modeling, and a better understanding of the gap between theoretical model design and practical training constraints.

This project successfully demonstrates a character-level Transformer language model implemented in TensorFlow. Even with limited training, the model showed promising performance and learned meaningful patterns in text data. The experience provided a comprehensive understanding of the end-to-end machine learning workflow and highlighted the importance of computational resources, training strategies, and model monitoring. With further training and improvements, the model could generate higher-quality and more coherent text.